Object Oriented Programming (CSEG1044)
Unit 05 – Lecture 01 Notes
Collections Framework Overview + Iteration + List

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Core interfaces: `Iterable`, `Collection`, `Iterator` . . . 3

7.2 List: ArrayList vs LinkedList . . . 4

7.3 Sorting: Comparable vs Comparator . . . 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: list + iterator + sorting **5**

11 Demo / Observation Task **5**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **6**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Collections Framework and Wrapper Classes
- **Main new ideas:** Core interfaces: `Iterable`, `Collection`, `Iterator`, List: ArrayList vs LinkedList, Sorting: Comparable vs Comparator
- **Course thread:** Use Collections Framework Overview + Iteration + List to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

2 Lecture Overview

The Collections Framework gives you standard, reusable data structures in Java. Instead of creating arrays manually and writing custom logic, you use:

- `List` for ordered sequences,
- `Set` for unique values,
- `Map` for key-value lookups.

This lecture focuses on **lists**, **iteration**, and **sorting**.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Generics**: Collections are safer and clearer once typed containers are already familiar. Main reference: `Unit 04 / Lecture 01 slides/notes`.
- **Comparison logic**: Sorting becomes easier once small comparison behavior is familiar. Main reference: `Unit 04 / Lecture 03 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Explain `Iterable`, `Collection`, and `Iterator`.
- Choose `ArrayList` vs `LinkedList` for common use cases.
- Sort a list using `Comparable` and `Comparator` (with lambdas).

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Core interfaces: `Iterable`, `Collection`, `Iterator`* → *List: ArrayList vs LinkedList* → *Sorting: Comparable vs Comparator.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Collection**: group of elements (root interface for many collections).
- **Iterable**: can be used in for-each loop (provides `iterator()`).
- **Iterator**: cursor used to traverse a collection safely.
- **List**: ordered collection with duplicates allowed and index-based access.
- **Comparable**: natural ordering (`compareTo`).
- **Comparator**: custom ordering (`compare`).

7 Detailed Notes

7.1 Core interfaces: `Iterable`, `Collection`, `Iterator`

Iterable:

- Any `Iterable` can be used with the enhanced for loop.

Iterator:

- Provides `hasNext()`, `next()`, and optional `remove()`.
- Use iterator when you need to remove elements while looping.

Common pitfall. Removing from a list inside a for-each loop can throw `ConcurrentModificationException`. If removal is required, switch to an `Iterator`.

7.2 List: ArrayList vs LinkedList

ArrayList:

- Backed by a dynamic array.
- Fast random access: `get(i)` is efficient.
- Good for most use cases: iterate a lot, access by index.

LinkedList:

- Nodes with links (doubly linked list).
- Slower random access (must traverse nodes).
- Useful when you frequently add/remove at ends and use it as a queue/deque.

Rule of thumb. Use `ArrayList` by default unless you have a clear reason to use `LinkedList`.

7.3 Sorting: Comparable vs Comparator

Comparable:

- implemented by the class itself (natural ordering).
- example: sort students by roll number.

Comparator:

- separate ordering logic, reusable and flexible.
- example: sort students by CGPA descending.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

8 Running Project Connection

Use Collections Framework Overview + Iteration + List to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Ignoring generics and using raw collections.
- Changing a collection unsafely while iterating.
- Choosing a List when the real requirement is uniqueness.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

10 Worked Example: list + iterator + sorting

```java
import java.util.*;

class Student {
    private final String rollNo;
    private final double cgpa;

    Student(String rollNo, double cgpa) {
        this.rollNo = rollNo;
        this.cgpa = cgpa;
    }

    public String getRollNo() { return rollNo; }
    public double getCgpa() { return cgpa; }
    public String toString() { return rollNo + ":" + cgpa; }
}

public class Demo {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("21CS1002", 9.1));
        list.add(new Student("21CS1001", 8.2));
        list.add(new Student("21CS1003", 3.9));

        // Remove weak students safely using Iterator
        Iterator<Student> it = list.iterator();
        while (it.hasNext()) {
            if (it.next().getCgpa() < 4.0) it.remove();
        }

        // Sort by CGPA descending using Comparator + lambda
        list.sort((a, b) -> Double.compare(b.getCgpa(), a.getCgpa()));
        System.out.println(list);
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Core interfaces: `Iterable`, `Collection`, `Iterator`* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Why do we use `Iterator.remove()` instead of `list.remove(...)` inside a for-each loop?

   Answer: to avoid concurrent modification issues; iterator removal is safe during traversal.

2. Which list is better for random access by index?

   Answer: `ArrayList`.

3. What is the difference between `Comparable` and `Comparator`?

   Answer: `Comparable` defines natural order inside class; `Comparator` defines external/custom order.

13 Self-Study Practice (no solutions)

1. Create a list of integers and remove all odd numbers using an iterator.
2. Sort a list of strings by length using a comparator lambda.
3. Implement `Comparable` in a class `Book` to sort by title.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why does Java provide a Collections Framework?

Sample answer: It provides standard, reusable data structures and algorithms, reducing bugs and improving productivity compared to writing custom structures every time.

Q2. When should you prefer `Comparator` over `Comparable`?

Sample answer: Prefer `Comparator` when you need multiple sorting orders or when you cannot/should not modify the class to define natural ordering.

Q3. What decides between ArrayList and LinkedList in practice?

Sample answer: Access patterns, update patterns, readability, and real measured needs decide the choice.